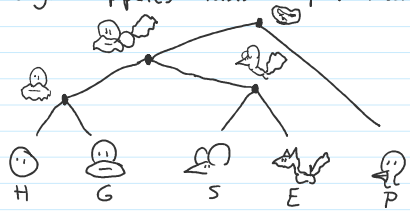
Phylogénétique

6 novembre 2024 cours 9

- Étude de relations évolutives entre objets
  - └ objets appelés taxa (pluriel de taxon)

ex:



ex: langues

Suppositions:
- évolution en arbre
- chaque taxon ancestral donne lieu à 2 taxa enfants (arbres binaire)
- feuilles = taxa contemporains (d'aujourd'hui)
  nœuds internes = taxa ancestraux inconnus

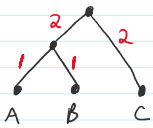
Entrée: feuilles (séquences, distances, ...)
Sortie: arbre
- └ étiquette sur les arêtes qui représente un temps d'évol.
- └ état des ancêtres

Approches:
1. par distances
2. par caractères
3. statistiques

1) Approches par distances

ex: 3 taxa A, B, C $D_{X,Y}$ = distance entre X et Y

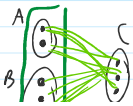
D	A	B	C
A	0	2	5
B	2	0	5
C	5	5	0



- Algorithme UPGMA
  - └ unweighted pair group with arithmetic mean

Entrée: matrice D (rangées = taxa, colonnes = taxa)
$D_{X,Y}$ = dist. X et Y symétrique

1) choisir A, B qui minimisent $D_{A,B}$
2) ajouter un nouveau taxon qu'on va appeler (A∪B)
   → ajouter une rangée + colonne à D pour (A∪B)
3) retirer les rangées A et B
4) pour chaque taxon C restant calculer $D_{(A\cup B),C}$

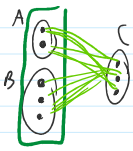


ex: D

	A	B	C	D	E	A∪B	D∪E
A	0	2	4	6	6		
B		0	4	6	6		
C			0	6	6	4	6
D				0	4	6	
E					0	6	

A∪B
∪C

4) pour chaque taxon C restant calculer $D_{(A\cup B),C}$

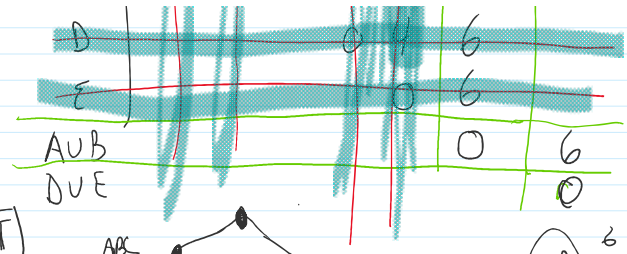


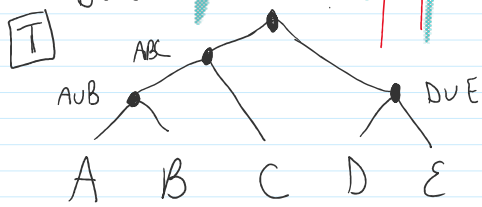
$D_{(A\cup B),C}$ = moyenne des dists entre les taxa $(A\cup B)$ et C

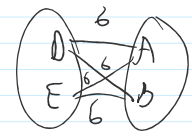
$$= \frac{1}{\#\text{dists}} \cdot \sum_{x\in(A\cup B)} \sum_{y\in C} D_{x,y}$$

$$= \frac{1}{|A\cup B|\cdot|C|} \cdot \sum_{x\in(A\cup B)} \cdot \sum_{y\in C} D_{x,y}$$

5) goto 1) jusqu'à ce qu'il reste 1 taxon







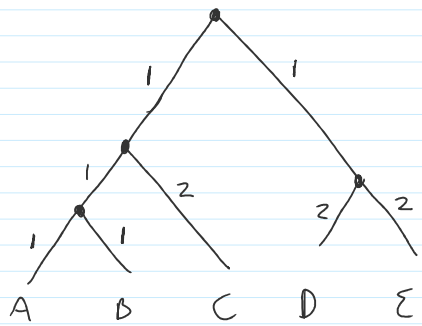
$\frac{6+6+6+6}{4}$

Q: peut-on étiqueter les branches de façon à ce que dans l'arbre T,

$$dist_T(X,Y) = D_{X,Y}$$

$dist_T(X,Y)$ = somme des poids des branches sur le chemin X,Y dans T

	A	B	C	D	E
A	0	2	4	6	6
B		0	4	6	6
C			0	6	6
D				0	4
E					0



Un tel étiquetage fonctionne si la distance D (matrice) est ultramétrique.

- Une distance D est ultramétrique s'il existe un arbre T qui satisfait les conditions:

① Pour tous taxa u,v, $D_{u,v} = dist_T(u,v)$

② Soit r la racine de T. Alors $\forall$ taxa u,v,

$$dist_T(r,u) = dist_T(r,v)$$

<u>Théorème</u>: si D est ultramétrique, alors UPGMA reconstruit un arbre capable de satisfaire ① et ②

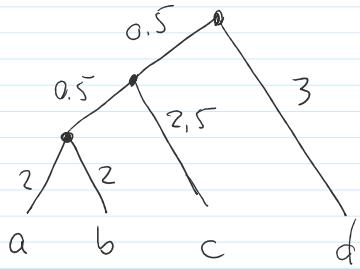
Dans les données réelles, les distances ne sont jamais ultramétrique.

Une autre approche, Neighbor-Joining, permet d'alléger ces conditions. Cette approche se concentre sur les <u>distances additives</u>.

• Une distance D est <u>additive</u> s'il existe un arbre T tel que
① $\forall u,v$, $D_{u,v} = dist_T(u,v)$.

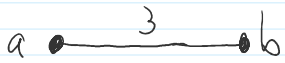
Q: comment vérifier qu'une distance est additive?

	a	b	c	d
a	0	4	5	6
b		0	5	6
c			0	6
d				0

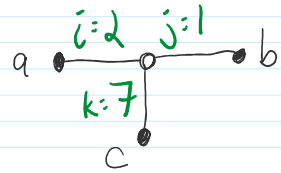


	a	b	c	d
a	0	3	9	8
b		0	8	8
c			0	5
d				0

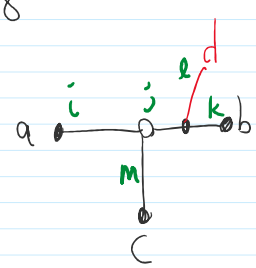
∃? arbre additif? Cet arbre n'a pas besoin d'être enraciné.

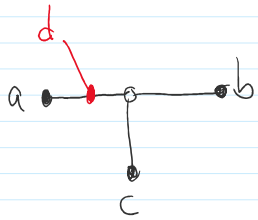


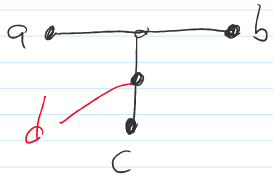
$i+j = 3$
$i+k = 9$
$j+k = 8$



$i+j+l = 6$
$i+j+k = 3$
⋮



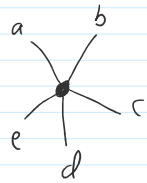




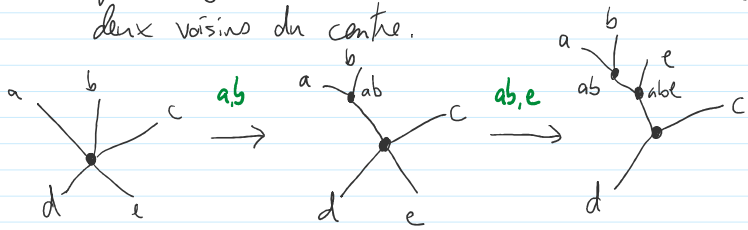
• Remarque: si une distance D est additive, alors il existe un seul arbre unique qui peut représenter les distances.

• L'algo Neighbor-Joining trouve un arbre additif s'il existe.
S'il n'existe pas, l'algo approxime "bien" les distances.

Idée: • démarrer d'un arbre étoile
• raffiner cet arbre une étape à la fois, en joignant successivement



• raffiner cet arbre une étape à la fois, en joignant successivement deux voisins du centre.



• On arrête quand le centre a 3 voisins.
↳ arbre non-enraciné.

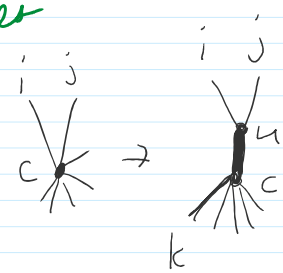
Q: comment choisir les voisins du centre à joindre? (Compliqué)

algoNJ(D)
  soit T l'arbre étoile avec 1 feuille par taxon
  c ← centre de T
  tant que c a plus que 3 voisins
    parmi les m voisins de c, choisir
    les deux voisins i, j qui minimisent

$$\underbrace{(m-2)D_{i,j}}_{i,j \text{ proches}} - \underbrace{\left(\sum_{k=1}^{m} D_{i,k} + \sum_{k=1}^{m} D_{j,k}\right)}_{i \text{ et } j \text{ loins des autres}}$$

    Joindre i, j en créant un nouveau noeud u
    Mettre à jour $D_{u,k}$ $\forall k \neq i, j$

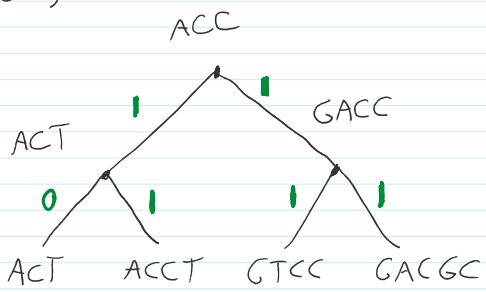
$$D_{u,k} = \frac{1}{2}\left[D_{i,k} + D_{j,k} - D_{i,j}\right]$$



Approches par caractères

Entrée: séquences $S_1, S_2, \ldots, S_m$
Sortie: arbre T avec
- feuilles étiquetées par les S_i
- tout noeud interne v soit étiqueté par une séquence $s(v)$
- minimiser la somme des coûts sur les branches

une séquence $S(v)$

- minimiser la somme des <u>coûts</u> sur les branches (maximiser un score)

ex: S_1 ACT
S_2 ACCT
S_3 GTCC
S_4 GACGC